Shapes-VQA Dataset: How to Use It

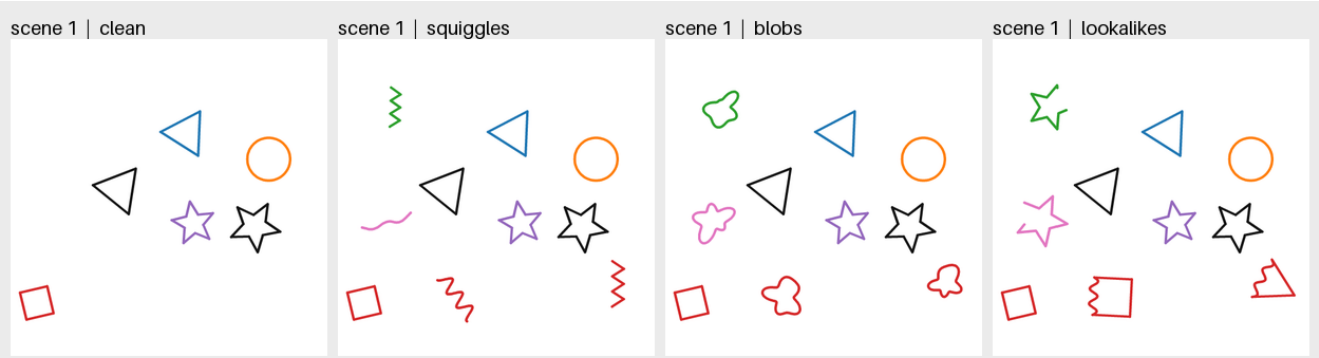
ATML project | huggingface.co/datasets/atml-shapes-vqa/shapes-vqa-distractors | questions: ask Mohid

What it is

A synthetic VQA dataset for testing whether visual clutter makes a VLM worse at counting and naming shapes. There are **500 scenes**. Each scene is a random layout of 3-7 named shapes (circle, triangle, square, star), drawn in **4 versions**: **2,000 images** in total (512 x 512).

Version	Difficulty	What is added to the named shapes
clean	0	nothing
squiggles	1	scribble lines (waves, zigzags, meanders)
blobs	2	irregular closed blobs that look like no named shape
lookalikes	3	named shapes with a defect: a gap in the outline, or one wavy side

> **Key idea:** the named shapes are pixel-identical in all 4 versions of a scene, so **all 4 versions have the same correct answer**. If accuracy drops from *clean* to *lookalikes*, the distractors caused it.



Every image uses the same question. The answer always lists all 4 classes in this order, including zeros:

```
circle: 1, triangle: 0, square: 2, star: 1
```

Splits are by scene, so all 4 versions of a scene are always in the same split: **train** 1,400 images (350 scenes), **validation** 300 (75), **test** 300 (75).

Useful columns

Column	Meaning
`image_id`	e.g. scene_0042_blobs. The key for your predictions
`scene_id`	same for all 4 versions of a scene. Use it to compare versions
`version`, `difficulty`	clean / squiggles / blobs / lookalikes = 0 / 1 / 2 / 3
`count_circle` ... `count_star`, `count_total`	true counts as integers
`num_distractors`	0 for clean, otherwise 2, 4 or 6
`objects`	JSON list of every object (target or distractor) with shape, colour, center, radius and bbox. Useful for finding which image patches hold distractors

Step 1. Get access (once)

1. Join the **atml-shapes-vqa** organization on Hugging Face (Mohid will send you an invite). Make a free account at huggingface.co first if you don't have one.

2. Go to **huggingface.co/settings/tokens**, click **Create new token**, choose type **Read**, and copy it.

3. In a terminal:

```
pip install datasets huggingface_hub pillow
hf auth login          # paste your token (older versions: huggingface-cli login)
```

Step 2. Load the data

```
from datasets import load_dataset
ds = load_dataset("atml-shapes-vqa/shapes-vqa-distractors")
test = ds["test"]            # 300 images = 75 scenes x 4 versions
ex = test[0]
ex["image"]       # PIL image, 512 x 512
ex["question"]    # the prompt (identical for every image)
ex["answer"]      # "circle: 1, triangle: 0, square: 2, star: 1"
ex["version"]     # clean / squiggles / blobs / lookalikes
lookalikes = test.filter(lambda e: e["version"] == "lookalikes")   # one version only
```

Step 3. Run your model and save its answers

Save a CSV with two columns, `image_id` and `prediction` (the model's raw text answer):

```
import csv
def ask_model(image, question):
    ...   # your VLM here (Qwen2-VL, LLaVA, InternVL, ...). Return its text answer.

with open("preds_mymodel.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["image_id", "prediction"])
    for ex in test:
        w.writerow([ex["image_id"], ask_model(ex["image"], ex["question"])])
```

Step 4. Score it

```
# once: download the scoring script
hf download atml-shapes-vqa/shapes-vqa-distractors vqa_scoring.py --repo-type dataset --local-dir .

# score your predictions (it fetches the correct answers from Hugging Face by itself)
python vqa_scoring.py --predictions preds_mymodel.csv --split test
```

You get one row per version: **exact** (all 4 counts right), **total** (total count right), per-class accuracy, **abs_err** (average count error), **overcnt** (how often the model counted distractors as real shapes) and **parsed** (answers it could read). Below that is a **paired comparison against clean** on the same scenes, with a McNemar p-value. Add `--out scores.csv` to save per-image results. The parser accepts messy answers like *"2 circles"* or *"**Circle**: two"*.

Rules of thumb

• **Report results on the test split.** Use train / validation for anything you tune or train (e.g. the disentanglement method).

• **Use the question as given**, so the 4 versions stay comparable. If you try a different prompt, run it on all 4 versions.

• **Compare versions on the same scenes** (the scorer does this for you).

• The dataset is read-only for members. If something looks wrong, tell Mohid; don't edit your local copy.